Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Исследование сходимости степенных рядов Тейлора для элементарных функций»

Выполнил:
студент группы 3825Б1ПМии1
Огнев Роман Андреевич

Проверила:

Бусько П.В.

Нижний Новгород
2026

Содержание

Введение 3
Постановка задачи 4
Руководство пользователя 5
Описание программной реализации 5
Результаты экспериментов 9
Заключение 12
Литература 13
Приложение 14

Введение

В этой программе я исследую сходимость степенных рядов Тейлора для простых функций. Необходимо узнать погрешность при вычислении рядов разными способами. По итогу необходимо определить наиболее точный способ подсчёта степенных рядов Тейлора. Также овладеть новыми способами вывода информации.

Постановка задачи

Цель: «Научиться вычислять значения математических функций (Sin, Cos, Log, Exp) с помощью разложения рядов Тейлора на языке «С». Сравнить два способа поиска суммы для конкретных функций»

Задачи:

1. Изучить формулы разложения функций и вывести рекуррентные соотношения.
2. Разработать многофайловый проект.
3. Написать единую функцию, которая может вычислять разные функции в зависимости от передаваемых параметров.
4. Реализовать вывод в формате `.CSV` (от англ. ***Comma-Separated Values*** — «значения, разделённые запятыми») — текстовый формат для хранения табличных данных.
5. Проанализировать полученные данные, выяснить точность измерений и сделать вывод о сходимости рядов Тейлора.

Руководство пользователя

После запуска программы пользователю выдается консольное приложение, в котором нет необходимой информации. Для получения необходимой информации, необходимо обратиться к файлу Microsoft Excel, в котором храниться таблица с данными, полученные в результате выполнения программы.

```
The results are stored in the "lab.results" file.
C:\Users\рома\source\repos\Lab_work_2semestr(Taylor)\x64\Release\Lab_work_2semestr(Taylor).exe (процесс 9256) завершил р
аботу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

Рисунок 1 Вывод консольного приложения.

Найти файл с таблицей можно в файле, где храниться сам код.

рома > source > repos > Lab_work_2semestr(Taylor)

Имя	Дата изменения	Тип	Размер
Lab_work_2semestr(Taylor)	29.03.2026 19:29	Папка с файлами	
x64	29.03.2026 19:27	Папка с файлами	
Lab_work_2semestr(Taylor).sln	11.03.2026 23:27	Visual Studio Solut...	2 КБ

Рисунок 2 Путь хранения таблицы с результатами.

рома > source > repos > Lab_work_2semestr(Taylor) > Lab_work_2semestr(Taylor) >

Имя	Дата изменения	Тип	Размер
x64	29.03.2026 19:27	Папка с файлами	
lab.results	29.03.2026 19:29	Файл Microsoft Ex...	1 КБ
Lab_work_2semestr(Taylor).cpp	29.03.2026 19:27	C++ Source	1 КБ
Lab_work_2semestr(Taylor).vcxproj	12.03.2026 0:09	VC++ Project	7 КБ
Lab_work_2semestr(Taylor).vcxproj.filters	12.03.2026 0:09	VC++ Project Filte...	2 КБ
Lab_work_2semestr(Taylor).vcxproj.user	11.03.2026 23:27	Per-User Project O...	1 КБ
Taylor.cpp	29.03.2026 19:29	C++ Source	3 КБ
Taylor.h	26.03.2026 0:46	C/C++ Header	1 КБ

Рисунок 3 Таблица с результатами.

```
int main() {
    struct Xn d;
    int n = 4;
    float x = 2.0f;

    const char* Myfile = "lab.results.csv";
    prepare_csv(Myfile);

    int_struct(&d, n);
    compute_sin(&d, x);
    save_csv(&d, "SIN(x)", Myfile);
    cout << "The results are stored in the \"lab.results\" file.";
```

Рисунок 4 Функция main, в которой вводятся параметры для функции.

Для того, чтобы подсчитать другую математическую функцию, необходимо заменить ключевое слово «compute_...», где вместо трех точек может быть: sin, cos, ln, exp. Также пользователь может изменить количество членов ряда Тейлора (параметр n) и значение самого x.

В предпоследней строчке «SIN(x)» не несет в себе смысловой нагрузки, а лишь отображается в таблице Excel, чтобы было удобнее ориентироваться в таблице при большом количестве разных функций.

Описание программной реализации

Мой проект представляет собой много файловую программу, включающую в себя исходные файлы и файлы заголовков, которые содержат в себе прототипы функций.

Файлы заголовков:

- `Taylor.h` – прототип функции для подсчета рядов Тейлора.

Исходные файлы:

- `Main.cpp` – главный файл, содержит функцию `main()`, запуск программы.
- `Taylor.cpp` – файл, содержит внутри все рекуррентные функции, главную функцию по подсчету рядов Тейлора, функцию вывода данных и запись их в таблицу.

Функции:

1. `Int main()` – в ней написана основная программа. Координирует ввод данных пользователем, инициализирует структуры, последовательно вызывает вычислительные функции и экспортирует результаты.
2. `Void int_struct(struct Xn* xn, int n)` – функция инициализации. Выделяет динамическую память под массивы членов ряда, частичных сумм и значение абсолютных погрешностей.
3. `Void free_struct(struct Xn* xn, int n)` – функция очистки. Освобождает ранее выделенную память.
4. `Void compute_[sin, cos, ln, exp](struct Xn* xn, float x)` – группа вычислительных функций. Находит значение функции через разложение ряда Тейлора с использование рекуррентных соотношений. Каждый новый член ряда «a[i]» получается путем умножения предыдущего «a[i-1]» на коэффициент.
5. `Static float _[sin, cos, ln, exp](float x, int shag)` – вспомогательная функция для вычисления «a[i]», тот самый коэффициент, на который умножается предыдущий член ряда «a[i-1]».
6. `Void static universal(Xn* xn, float x, TaylorMode mode)`- основная расчетная функция, реализующая итерационный цикл суммирования членов ряда Тейлора. В зависимости от выбранного режима производит расчет по одной из двух стратегий:
    a. ***Прямое суммирование***: накопление результата от первого (наибольшего) члена ряда к последующему.

b. ***Обратное суммирование***: Предварительный расчет всех членов ряда с последующем сложением, начиная с последнего (наименьшего) значения к первому.

7. `Void prepare_csv(const char* filename)` – очищает файл для записи новых данных. Добавляет в него заголовки столбцов «Результат» и «Погрешность».
8. `Void save_csv(Xn* xn, const char* func_name, const char* filename)` – производит экспорт данных в назначенный файл в формате CSV, записывает в него значения промежуточных сумм и соответствующую им погрешность, рассчитанную относительно эталонного значения из библиотеки «`math.h`»

Как работают алгоритмы подсчета ряда Тейлора

Для минимизации вычислительных затрат и предотвращения переполнения памяти при расчете с факториалом я использую рекуррентный метод. Суть алгоритма заключается в том, что каждый последующий член ряда «a[i]» вычисляется на основе предыдущего члена «a[i-1]» через множитель. Выше данные программы описаны под цифрами 4,5. Ознакомиться с самими множителями можно в разделе Приложение («Приложение 1»). Данный подход позволяет работать программе с любым количеством итераций «n».

Результаты экспериментов

Мною было проведено исследование работы алгоритма при разных условиях. Основной целью стало проверить, как направление суммирование членов ряда влияет на итоговую точность и накопление ошибки при вычислении. Эксперименты помогли оптимизировать код и добиться стабильных результатов:

1. Динамические массивы успешно хранят в себе числа для любого количества итерации n.
2. Реализовано суммирование двумя способами, что помогло выявить наилучший, который накапливает наименьшую ошибку.
3. Написанные рекуррентные соотношения обеспечивают правильный расчет всех функций.
4. Результат успешно экспортируется в формат CSV, на основе которых построены графики.

Вывод результатов экспериментов в программе:

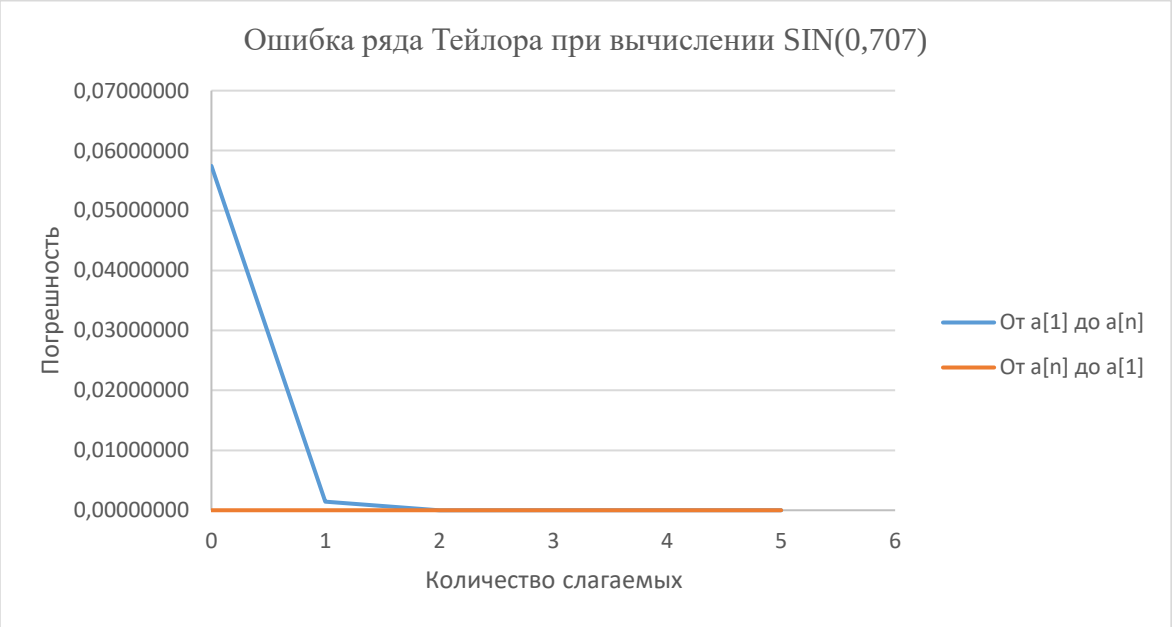

Figure 1График сравнения погрешностей SIN(x)

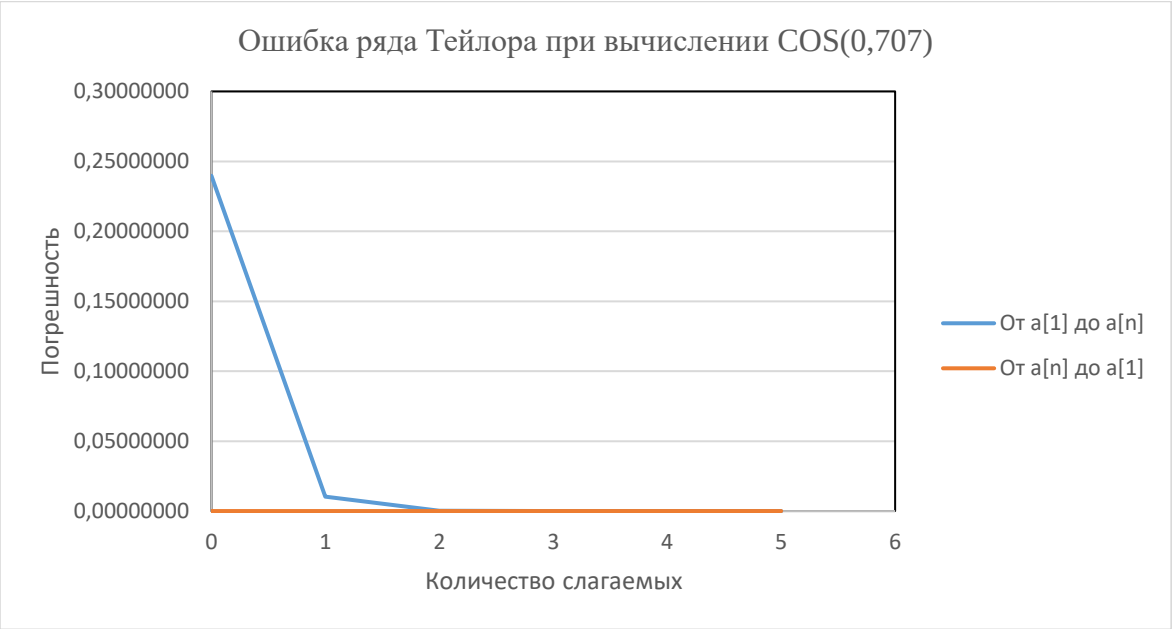

Figure 2 График сравнения погрешностей COS(x)

Для sin(x) и cos(x) наблюдается характерный ступенчатый график снижения погрешности. Прямой порядок суммирования обеспечивает быстрое уменьшение абсолютной погрешности, в то же время обратный порядок обеспечивает наименьшую ошибку вплоть до 3-й итерации. После порядки суммирования «выравниваются» по ошибке.

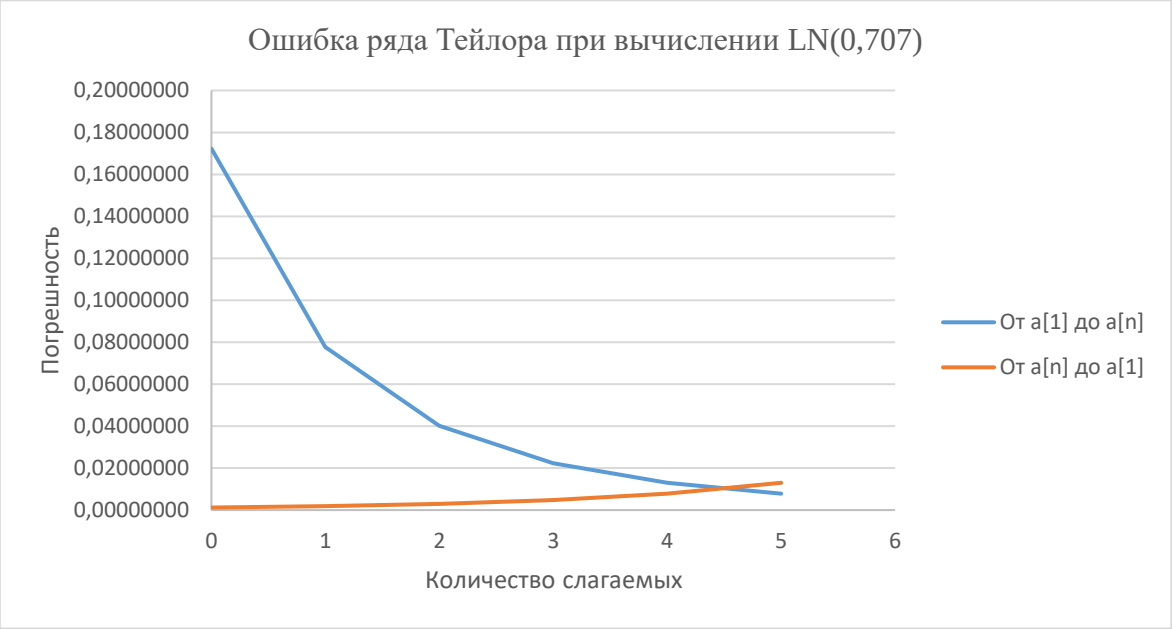

Figure 3 График сравнения погрешности LN(x)

В отличии от тригонометрических функций можно наблюдать более пологий спуск, что обусловлено более медленным убыванием членов ряда. Обратный порядок суммирования начинает с наименьшей ошибкой, но к 6-й итерации накапливает такую же ошибку, что и прямой порядок суммирования

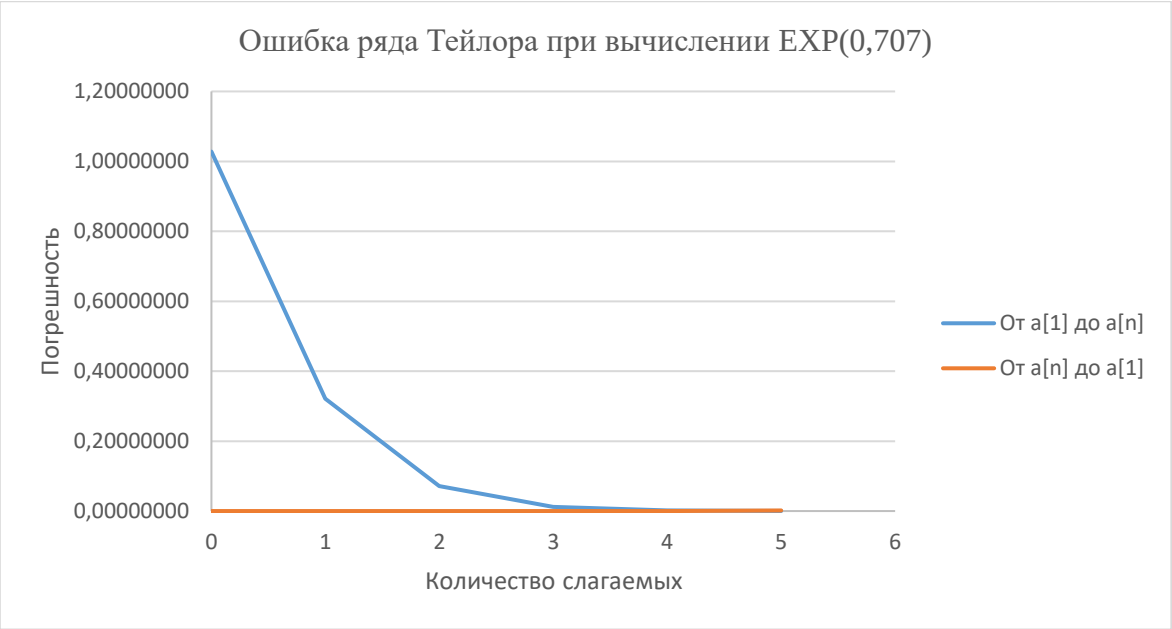

Figure 4 График сравнения погрешности EXP(x)

Наглядно показана высокая скорость сходимости, погрешность нивелируется уже после 6-й итерации. Обратный порядок суммирования обеспечивает минимальную погрешность.

Было проведено 8 экспериментов, результатами которых стали таблицы, на основе которых были построены графики изменения погрешностей вычисления рядов Тейлора в сравнении с идеальными значениями. По результатам экспериментов можно сделать несколько выводов:

1. При прямом суммировании значения результата «уточняются» сверху вниз. При обратном суммировании мы видим, что значение плавно поднимается с последней строчки на первую, что подтверждает то, что программа начинает заполнять таблицу с последнего x[i].
2. При дальнейшем подсчете значений, числа будут округляться до 0.0, не изменяя текущий результат.
3. При обратном подсчете малые числа складываются раньше, то есть их не поглощают большие числа, значит итоговый результат при таком подсчете будет точнее.
4. Выбор порядка сложений влияет на точность конечного результата, но спустя определённое количество итераций значения погрешностей сравниваются.

Заключение

Выполняя лабораторную работу, был реализован рабочий код на языке «C» для вычисления значений функций через ряды Тейлора. Экспериментальным путем было доказано, что с увеличением числа «n» сумма ряда всё точнее приближается к стандартным значениям из math.h. Обратный порядок суммирование точнее, нежели прямой, было доказано экспериментально. Использование рекуррентных соотношений позволило избавиться от дублирующего кода, также сделало программу более быстрой и эффективной. Благодаря разделённому коду можно внести незначительные правки для подсчёта других функций. В коде был реализован новый формат вывода результатов, который позволяет намного быстрее строить таблицы и графики для анализа значений рядов Тейлора.

Литература

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.
2. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. / М.: Издательский дом «Вильямс», 2011.
3. Язык программирования С | Керниган Брайан У., Ритчи Деннис М. - Издательский дом Вильямс, 2016.
4. Б.П.Демидович Сборник задач и упражнений по математическому анализу (Учебное пособие для вузов) – АСТ Астрель Москва, 2005.

Приложение

```
static float _sin(float x, int shag) {
      return (float)(-(x * x)/ ((2 * shag + 1) * (2 * shag)));
}
static float _cos(float x, int shag) {
      return (float)(-(x * x)/ ((2 * shag - 1) * (2 * shag)));
}
static float _ln(float x, int shag) {
      return (float)(-x * (shag) / (shag + 1));
}
static float _exp(float x, int shag) {
      return (float)(x / shag);
```

Приложение 1 Рекуррентное соотношения для sin, cos, ln, exp

```
void save_csv(Xn* xn, const char* func_name, const char* filename) {
      FILE* fp;
      fopen_s(&fp, filename, "a");
      if (fp != NULL) {
            fprintf(fp, "--- %s ---\n", func_name);
            for (int i = 0; i < (xn->n); i++) {
                  fprintf(fp, "%.8f;%.8f\n", (float)xn->sum[i], (float)xn-
>err[i]);
            }
            fprintf(fp, "\n");
            fclose(fp);
      }
}
```

Приложение 2 Функция для заполнения файла данными.